Architecture 1001: x86-64 Assembly Pass1Parameter.c

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Pass1Parameter.c

Adding a single argument

```c
int func(int a){
    int i = a;
    return i;
}
int main(){
    return func(0x11);
}
```

```
func:
0000000140001000  mov         dword ptr [rsp+8],ecx
0000000140001004  sub         rsp,18h
0000000140001008  mov         eax,dword ptr [rsp+20h]
000000014000100C  mov         dword ptr [rsp],eax
000000014000100F  mov         eax,dword ptr [rsp]
0000000140001012  add         rsp,18h
0000000140001016  ret
main:
0000000140001020  sub         rsp,28h
0000000140001024  mov         ecx,11h
0000000140001029  call        func (0140001000h)
000000014000102E  add         rsp,28h
0000000140001032  ret
```



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!



STOP

&&

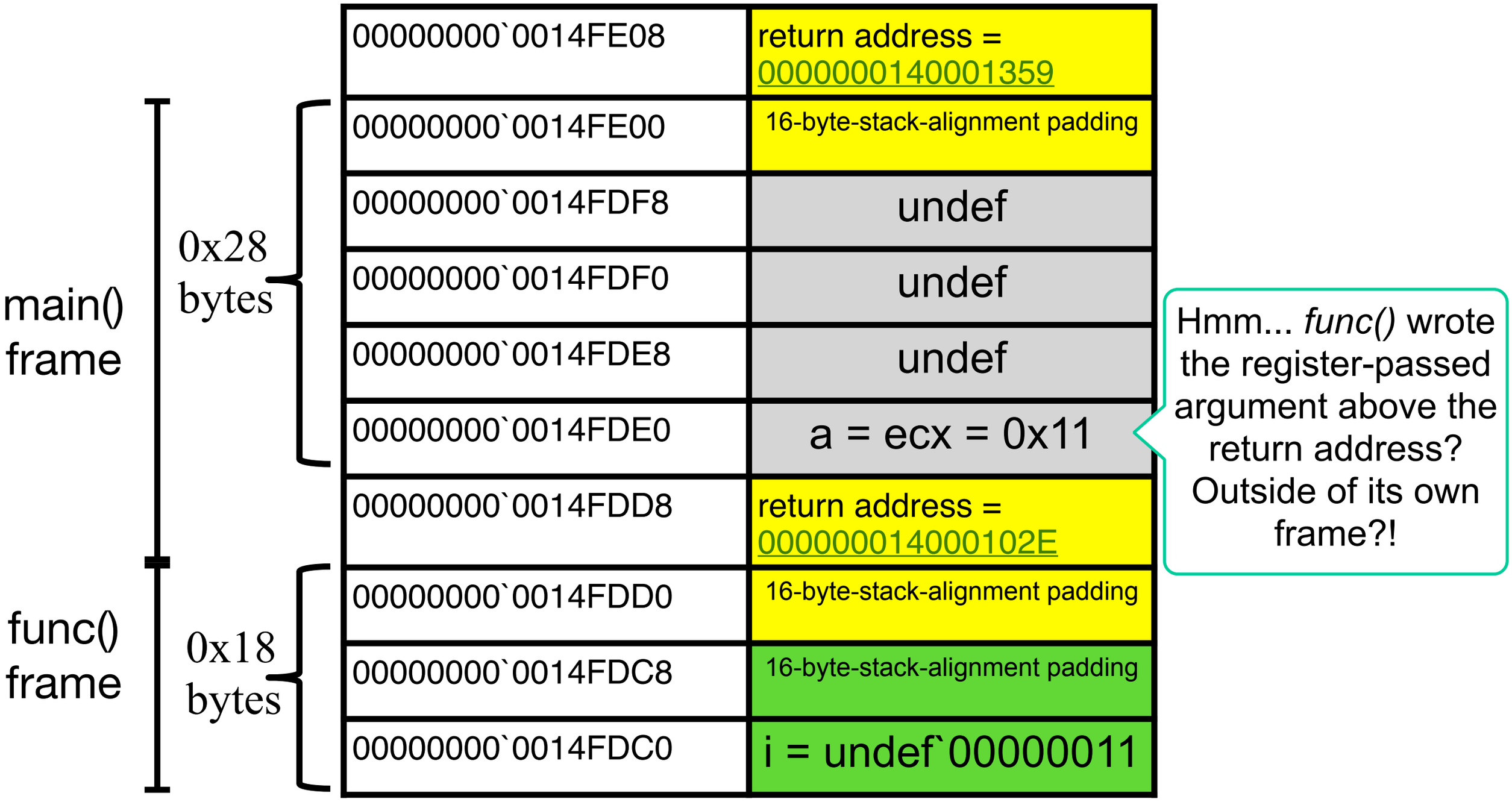
STEP THROUGH

THE ASSEMBLY

&&

DRAW A STACK DIAGRAM!

The stack looks like this at line 0000000140001000F in func():



Pass1Parameter.c takeaways

- Something very interesting is going on with the stack!
- The value which is passed in a register is then still being stored on the stack…doesn't that kind of defeat the speed benefit of passing arguments in registers?

```
int func(int a){
    int i = a;
    return i;
}
int main(){
    return func(0x11);
}
```

```
func:
0000000140001000  mov         dword ptr [rsp+8],ecx
0000000140001004  sub         rsp,18h
0000000140001008  mov         eax,dword ptr [rsp+20h]
000000014000100C  mov         dword ptr [rsp],eax
000000014000100F  mov         eax,dword ptr [rsp]
0000000140001012  add         rsp,18h
0000000140001016  ret
main:
0000000140001020  sub         rsp,28h
0000000140001024  mov         ecx,11h
0000000140001029  call        func (0140001000h)
000000014000102E  add         rsp,28h
0000000140001032  ret
```